$Malloc\ Lab$

计14 汪隽立 2021012957

实验目的

- 学习动态内存分配的基本原理，了解分配器的要求与目标。
- 学习显式分配器在动态内存分配中可能遭遇的问题，并尝试解决这些问题。
- 综合运用所学知识实现一个简单的分配器。实现与C语言中`malloc`，`free`，`realloc`功能一致的函数。
- 复习Linux系统的指令运用，以及-ssh指令的基本用法。

实验原理

堆

- 对于那些运行时才能确定大小的数据结构而言，程序员需要通过使用动态内存分配器（例如C语言的`malloc`）在运行时获取虚拟内存的空间。动态内存分配器维护着一个进程的虚拟内存区域，称为**堆**（heap）。
- 堆在一个进程的虚存中位于未初始化的数据段之后。堆由低地址向高地址生长，存在一个维护堆顶的指针`brk`。

内存分配

- 对于内存分配器，我们有如下要求：
  - 能够处理任意请求序列
  - 高速
  - 块与块之间不冲突
  - 满足对齐要求（e.g. 16字节）
  - 不修改已分配的块
- 在这些要求的基础上，我们有两个评判的标准：
  - 吞吐率（Throughput）：单位时间内完成的请求数。

    如：1s内完成了5000次malloc与5000次free操作，则吞吐率为10000 (operations/s)。
  - 内存峰值利用率（Peak memory utilization）：

    给定n个分配和释放请求的某种顺序$R_1, R_2, \ldots, R_k, \ldots, R_{n-1}$，在完成请求$R_k$的分配后，记聚集有效载荷为$P_k$。那么，前$k+1$个请求的峰值利用率即为

    $$U_k = \frac{max_{i \le k} P_i}{H_k} \quad （H_k\text{单调不减，为堆的大小}） \tag{1}$$

    我们的目标即让U_{n-1}最大
  - 最终的评判公式为$P = wU + (1-w)min(1, \frac{T}{T_{libc}}), w = 0.6$

分配器带来的问题

- 碎片化

    - 内部碎片化：块内未被利用的内存
    - 外部碎片化：块与块之间未被利用的内存
- 寻找合适块的时间：最坏情况为线性
- 内存块自身需要维护更多的信息：自身大小 / 是否被分配
- 在一些极端例子下，空间利用率可能相当低
- 其他

实验过程

为了最好地解决上面提到的问题，内存分配器最终使用了**显式空闲列表 + 分离适配 + best fit**的实现方式。在此之前，内存分配器也经历了如下版本：

隐式空闲链表

总的来说，隐式空闲链表在吞吐率和内存利用率都表现得最差，然而其组织内存块的数据结构以及合并空闲块的方式将被一直沿用下去。

在实现上，主要参考了CSAPP上对隐式空闲链表的实现。此外，也没有单独设计`mm_realloc`，即`mm_realloc`仅为简单地调用`mm_init`和`mm_free`。

数据结构

图 9-35　一个简单的堆块的格式

一个内存块除了有效载荷（payload）以外，还需要头部与尾部记载其信息。这里我们令图9-35中的**填充**为尾部，记载的信息与头部完全一致。

由于内存要求16字节对齐，所以块大小的低4bit一定为0。我们可以利用这4位记载块的分配信息。具体来说，最低位为0说明该块没有被分配，最低位为1说明该块被分配了。为了操作方便，分配器参考CSAPP设置了如下的宏，以便查询/修改头部及尾部的信息：

```
/* 将大小与占用信息整合到一个word中 */
#define PACK(size, alloc) ((size) | (alloc))

/* 对位于地址p的一个word进行读写 */
#define GET(p)            (*(unsigned long *)(p))
```

```
#define PUT(p, val)        (*(unsigned long *)(p) = (val))

/* 获取指针p的首字信息 */
#define GET_SIZE(p)        (GET(p) & ~0xf)
#define GET_ALLOC(p)       (GET(p) & 0x1)

/* 计算一个块的header或footer地址 */
#define HDRP(bp)           ((char *)(bp) - WSIZE)
#define FTRP(bp)           ((char *)(bp) + GET_SIZE(HDRP(bp)) - DSIZE)
```

有了头部和尾部的信息后，从一个内存块抵达前一个或后一个内存块也变得方便，只需加上或减去该块的大小即可。

```
/* 计算前一个或后一个块的地址 */
#define NEXT_BLKP(bp)      ((char *)(bp) + GET_SIZE(((char *)(bp) - WSIZE)))
#define PREV_BLKP(bp)      ((char *)(bp) - GET_SIZE(((char *)(bp) - DSIZE)))
```

总的来说，隐式空闲链表将内存块组织为空间上连续的链表。每个块可以通过自己头部及尾部记录的信息抵达前一个或后一个块。直观地看，堆的结构为：

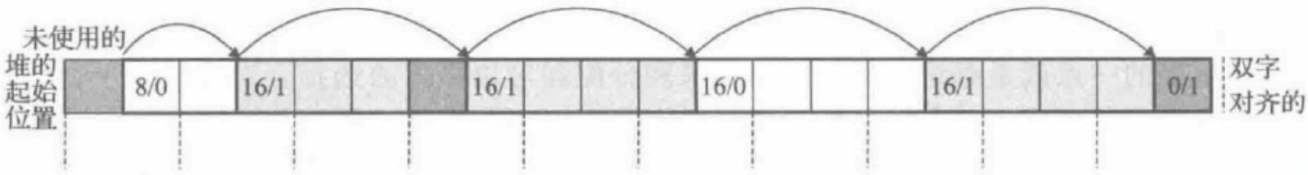


图 9-37　分割一个空闲块，以满足一个 3 个字的分配请求。阴影部分是已分配块。没有阴影的部分是空闲块。头部标记为(大小(字节)/已分配位)

合并(coalesce)

面对如上的数据结构，当遇到有多个空闲块在物理地址上连在一起时，我们应当将他们合并。可以想像，一共有四种情况：

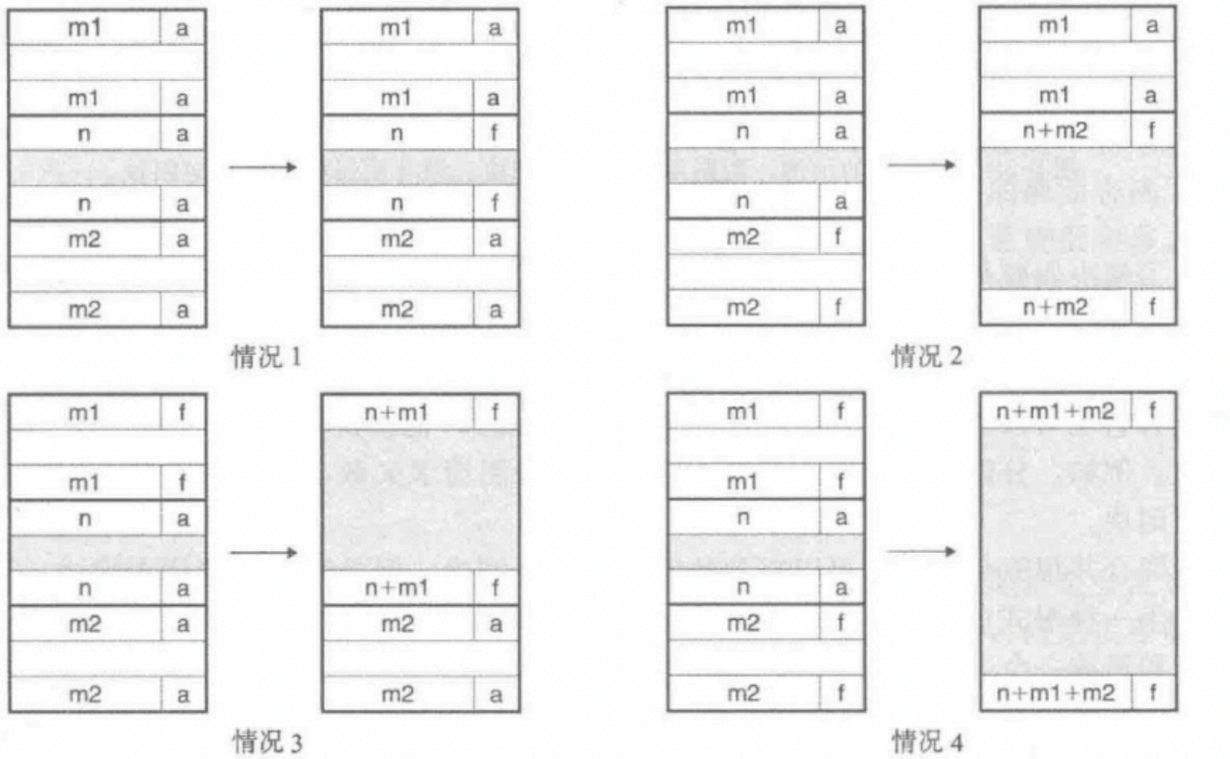


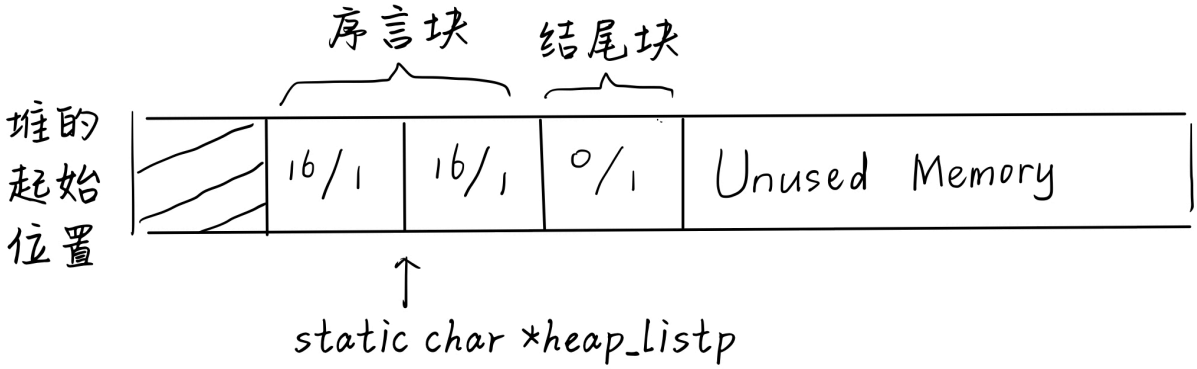
图 9-40 使用边界标记的合并(情况 1：前面的和后面块都已分配。情况 2：前面块已分配，后面块空闲。情况 3：前面块空闲，后面块已分配。情况 4：后面块和前面块都空闲)

其中情况2，3，4都改变了空闲块的实际分布。每次**释放**或者**开辟新的内存块空间**时，我们都需调用`coalesce`函数合并可能存在的空闲块。该函数返回指向新块的指针。

mm_init

有了清晰的数据结构概念后，我们便可以着手实际地设计堆的组成。参考CSAPP的实现方式，堆的起始位置是一个空白块。紧接着是一个**双字大小**的序言块，它被设为**已分配**。再紧跟着是一个结尾块，他的**大小被设为0且标记为已占用**。

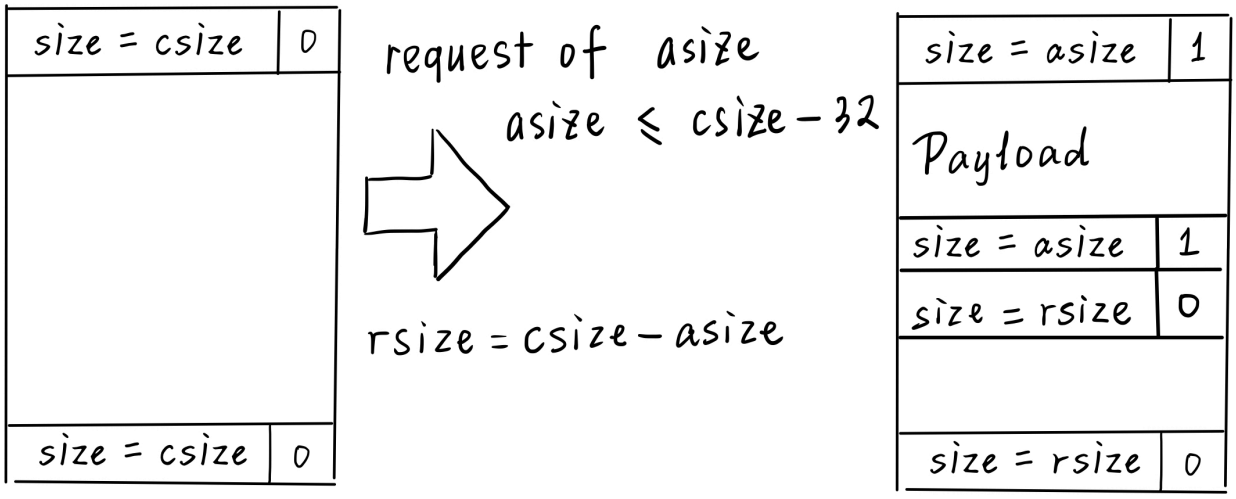
除此以外，我们还设置了静态全局变量`static char *heap_listp`，作为指向内存块区域开始位置的指针。具体来说，**初始的内存块区域**应当如下图：



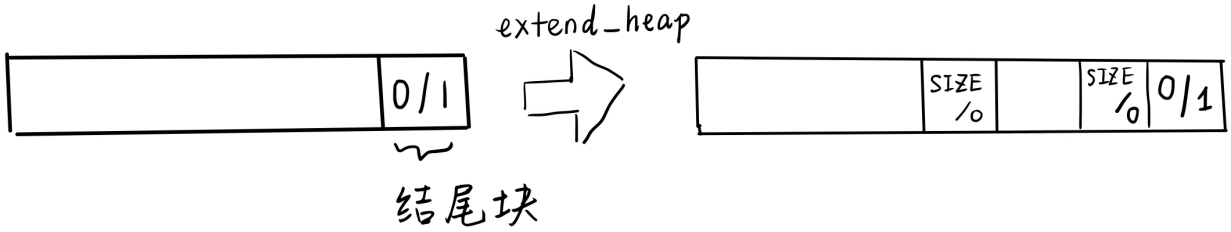
mm_malloc

`mm_malloc`完成的事有两件：将内存块封装好，并找到合适的位置将其安置。由于内存是16字节对齐的，且内存块的头部和尾部已经占用了16字节，故**内存块的最小大小为32字节**。在寻找合适的空闲块时，这里先采用初次适配（first fit）的方式。即：从序言块开始寻找，逐步寻找当前块的下一块，直到找到一个块，大小合适且为分配为止。

在安置内存空间时，被我们找到的空闲内存块可能空间过大。如果将它完全分配给请求，可能会导致内部碎片过大的问题。故在安置内存块（调用`place`函数）时，如果多出来的空间多于**32字节**，则将其分割。直观的例子如下图：



当不能搜索到合适的空闲块时，分配器就将开辟新的堆空间（调用`extend_heap`函数） 以供使用。为了提高效率，一次开辟的堆空间至少为`CHUNKSIZE`。



随后，将这个新开辟的块直接分配给需求，并进行相应的分割即可。

mm_free

只需将该块设置为未分配，并进行合并（coalesce）即可。

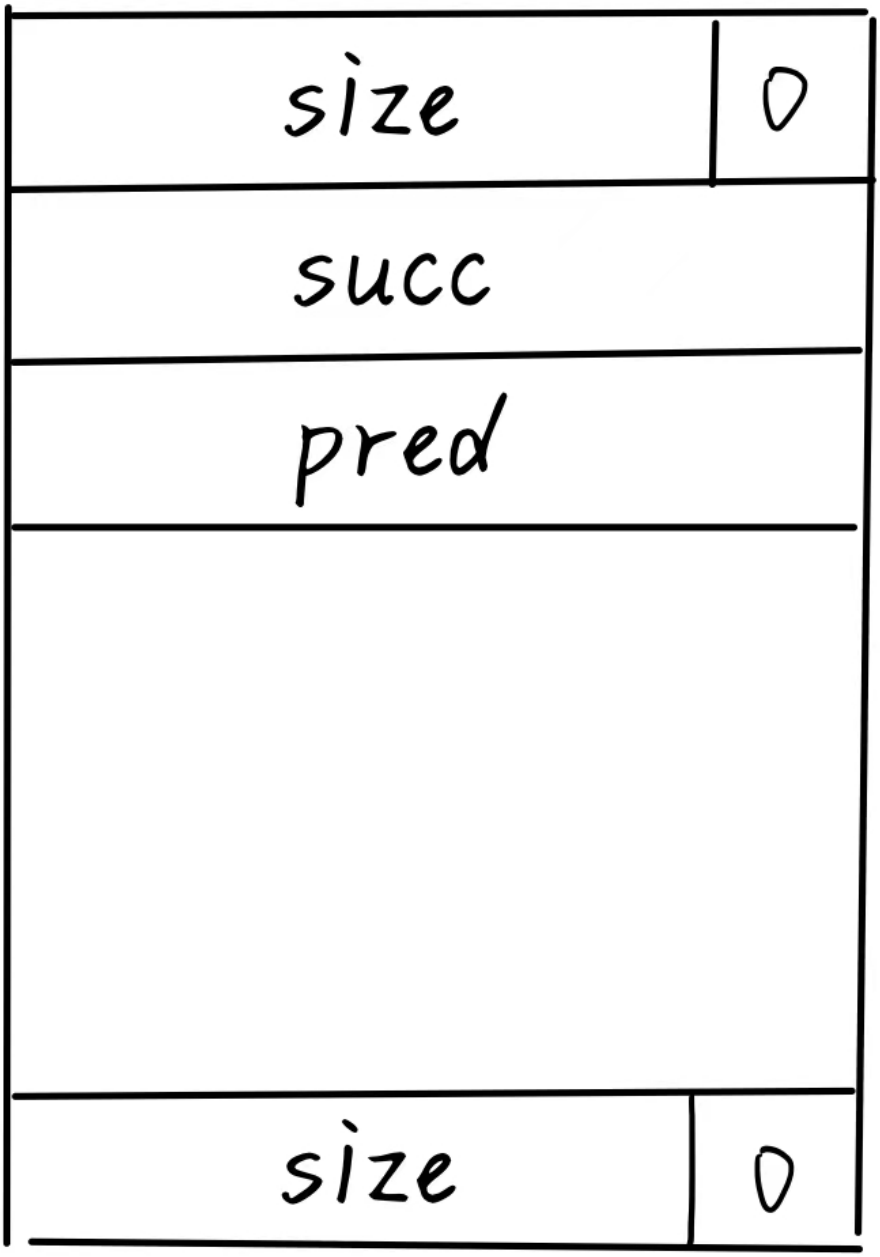
显式空闲链表

显式空闲链表的吞吐率和空间利用率都带来了一定的提升，不过离我们最终的目标还差了一些。

数据结构

与隐式空闲链表略有不同，所谓**显式的空闲链表，是指将所有的空闲块组成一个链表，已被占用的块不在这个链表中**。如此一来，搜索空闲块的时间大大减少——只要在显示空闲链表中找到空间大小合适的，直接放置即可。

每个**空闲块**多出了两部分内容：指向下一个/上一个空闲块的指针。而已被占用的块数据结构不变。



（注意：`succ`在前而`pred`在后！！！这与接下来分离链表的实现有关）

为了便捷地设置一个块的`succ`与`pred`信息，新增了如下宏：

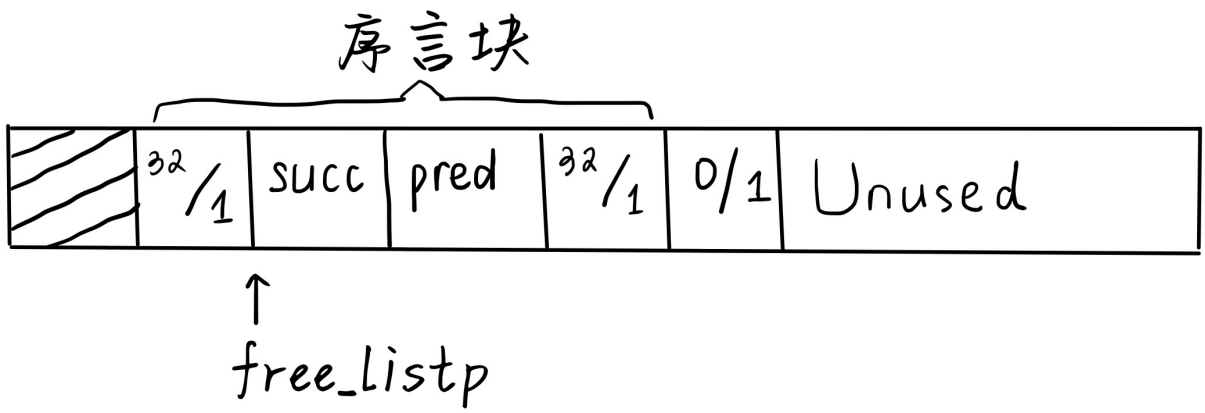
```
/* 显式空闲列表：获取一个空闲块的pred或succ */
#define SUCC_FREE(bp)      (GET(bp))
#define PRED_FREE(bp)      (GET((char *)(bp + WSIZE)))

/* 设置空闲块的pred或succ */
#define SET_SUCC(bp, val) PUT(bp, (unsigned long)val)
#define SET_PRED(bp, val) PUT((char *)(bp + WSIZE), (unsigned long)val)
```

显式空闲链表的插入和删除函数分别以`freelist_insert`和`freelist_remove`来实现。他们主要通过`SET_SUCC`和`SET_PRED`改变链表的局部结构。同时，新增了链表头指针`free_listp`，其指向实际与`heap_listp`一致。

mm_init

与隐式空闲链表的介绍一致，先列出初始的内存块结构：



可以看出，这里的链表头`free_listp`实际上是一个超起始，真正有效的内容应当从他的后继开始。当我们初次调用`extend_heap`之后，新生成的空闲块便会作为显式空闲链表的一员，并以`free_listp`为前驱插入。

此外，**序言块初始的`succ`应被设置为`NULL`**。

mm_free与合并(coalesce)

合并操作与隐式空闲链表的类似，需要注意的是两个块合为一个时，我们应当将被合并的块从链表中删除，再将新的块插入回去。

`mm_free`操作则是简单调用`coalesce`即可：找到链表的合适位置，将此空闲块插入其中。

mm_malloc

这里依然采用first fit的方式查找空闲块。只需从链表头`free_listp`的后继开始搜索，直到搜索到一个大小满足的块，或是搜索到了`NULL`。后者需要调用`extend_heap`进行堆拓展。

分离的空闲链表

> 就像我们已经看到的，一个使用单向空闲块链表的分配器需要与空闲块数量呈线性关系的时间来分配块。一种流行的减少分配时间的方法，通常称为分离储存（segregated storage），就是维护多个空闲链表，其中每个链表中的块有大致相等的大小。一般的思路是将所有可能的块大小分成一些等价类，也叫做大小类（size class）。——CSAPP第三版，第604页

为了简单说明，我们假定大小类被分为$[32], (32, 64], (64, 128], \ldots, (2048, 4096], (4096, \infty)$几类（出于调试的过程，实际程序的实现比这要略复杂一些）。

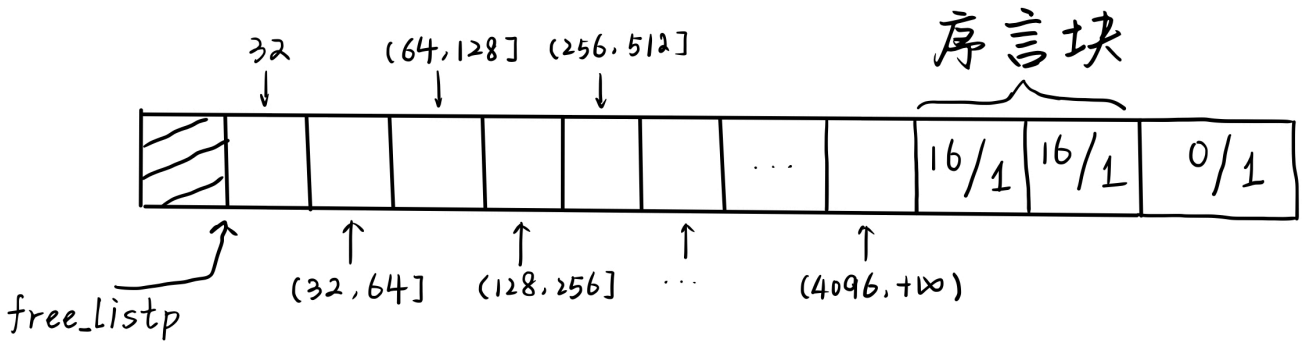
数据结构

块的结构与前一致，没有任何改变。但链表的插入操作需要调整：每次插入空闲块都需要根据它的大小，将其插入到对应的等价类里。也即`freelist_insert`的接口发生了改变，多出了第二个参数`size_t size`。

此外，程序还注册了函数`size2offset`，用于将具体的`size`映射为它所在的等价类。

mm_init

还是一样，先给出初始状态的内存块示意图：



其中：每个标有其所对应的等价类的**八字节块**即为该等价类独有的链表的超起始。例如：$(32, 64]$这一块即是这个等价类的链表头。

这里运用了一些trick：如前，一个空闲块的`succ`信息被安置在它起始后面的8个字节。而对于这些链表头，我们并**不在意他们的头部及尾部信息，更不在意他们的前驱**。故这些链表头不需要32字节组织而只需要8字节。**这在兼顾了存储空间的同时，还做到了接口的统一。**

而对于序言块来说，实际上它不属于任何一个等价类中，进而不属于任何一个链表。故它自然也不需要`succ`和`pred`的信息，它更多地起到了“哨兵”的作用。

故我们的`mm_init`函数如下：

```
int mm_init(void) {

  if ((heap_listp = mem_sbrk((CATEGORIES + 5) * WSIZE)) == (void *)-1) {
    return -1;
  }

  PUT(heap_listp, 0);
  heap_listp += WSIZE;
```

```
  free_listp = heap_listp;
  for (int i = 0; i < CATEGORIES; i++) {
    PUT(free_listp + i * WSIZE, 0);
  }
  /* 对齐 */
  PUT(heap_listp + 13 * WSIZE, 0);
  /* 序言块 */
  PUT(heap_listp + 14 * WSIZE, PACK(DSIZE, 1));
  PUT(heap_listp + 15 * WSIZE, PACK(DSIZE, 1));
  /* 结尾块 */
  PUT(heap_listp + 16 * WSIZE, PACK(0, 1));

  return 0;
}
```

mm_free与合并(coalesce)

与前文介绍显式空闲链表一致，在释放一个内存块后，都需要按照他们的等价类将他们插入到对应的链表中。

mm_malloc

`mm_malloc`调用的`find_fit`函数略有不同。在分离的空闲链表里，我们需要从需求大小的等价类里开始寻找，并逐渐增加等价类，直到找到一个满足的块为止。

```
static void *find_fit(size_t asize) {
  int offset = size2offset(asize);
  for (int i = offset; i < CATEGORIES; i++) { /* 从当前的大小类开始寻找 */
    void *bp = free_listp + i * WSIZE;
    for (bp = SUCC_FREE(bp); bp != NULL; bp = SUCC_FREE(bp)) {
      if (asize <= GET_SIZE(HDRP(bp))) { /* 显式空闲列表：大小符合的块一定满足需求 */
        return bp;
      }
    }
  }
  return NULL;
}
```

其他的细节与显式空闲链表一致，包括放置目标块，拓展堆等操作。

至此，基本的`mm_malloc`，`mm_free`已经实现。接下来的时间大多用来改良分配器。

最佳适配(best fit)

最佳适配，是指在当前等价类对应的链表中**寻找满足内存需要的最小块**。如此，分配器带来的碎片（包括内部碎片和外部碎片）都将减少。这对于内存利用率有一定的提升。

在程序的实现中，对`freelist_insert`做出了一定调整，即链表中空闲块是按大小排列的。这样便不用修改`find_fit`接口而做到最佳适配的效果。

```c
static void freelist_insert(void *bp, size_t size) {
  int offset = size2offset(size);
  void *header = free_listp + offset * WSIZE;
  void *prev = header;
  void *next = SUCC_FREE(header);

  /* bestfit : 找到插入的合适位置 */
  for ( ; next != NULL; prev = next, next = SUCC_FREE(next)) {
    if (size <= GET_SIZE(HDRP(next)))
      break;
  }
  SET_SUCC(prev, bp);
  SET_PRED(bp, prev);
  SET_SUCC(bp, next);
  if (next) {
    SET_PRED(next, bp);
  }
}
```

设计mm_realloc

正如实验导读所说，要想`mm_realloc`获得好的性能，需要做出单独的设计。在这次实验中，做出了如下的设计：

- 当`size == 0`或者`ptr == NULL`时，简单调用`mm_free`和`mm_malloc`即可。
- 当`size`大于原有块的大小时，尝试和**（物理意义上）的上一块及下一块合并并分配使用**
  - 若原有块与下一块的大小加起来足够用，则直接将下一块并入当前块使用
  - 否则，若上一块与原有块的大小加起来足够用，则直接将上一块并入当前块使用
  - 否则，若上一块与原有块与下一块三块的大小加起来足够用，则直接把这三块并起来使用
  - 否则，再调用`mm_free`和`mm_malloc`
- 若`size`大于原有块的大小，且**原有块位于堆顶，那么直接拓展堆顶相应的大小**。

```c
void *mm_realloc(void *ptr, size_t size) {
  /* 简单调用mm_malloc与mm_free的情况 */
  if (ptr == NULL)
    return mm_malloc(size);

  if (size == 0) {
    mm_free(ptr);
    return NULL;
  }

  /* ptr != NULL && size > 0 */
  void *newptr;
  void *prev;
  void *next;
  size_t asize;
  size_t old_size = GET_SIZE(HDRP(ptr));
  size_t prev_size = GET_SIZE(HDRP(PREV_BLKP(ptr)));
```

```c
  size_t next_size = GET_SIZE(HDRP(NEXT_BLKP(ptr)));
  size_t sum_size = old_size + prev_size + next_size;

  if (size <= DSIZE)
    asize = 2 * DSIZE;
  else
    asize = ALIGN(size + DSIZE);

  if (asize <= old_size) { /* 新块大小比原先要小 */
    PUT(HDRP(ptr), PACK(old_size, 1));
    PUT(FTRP(ptr), PACK(old_size, 1));
    return ptr;
  }
  else { /* 新块的大小比原先要大 */
    prev = PREV_BLKP(ptr);
    next = NEXT_BLKP(ptr);
    if (GET_SIZE(HDRP(next)) == 0) { /* 当前块位于堆顶 */
      size_t extendsize = asize - old_size;
      if ((long)(mem_sbrk(extendsize)) == -1) { /* 堆顶空间不足以使用 */
          newptr = mm_malloc(size);
          memmove(newptr, ptr, old_size - WSIZE);
          mm_free(ptr);
          return newptr;
      }
      PUT(HDRP(ptr), PACK(asize, 1));
      PUT(FTRP(ptr), PACK(asize, 1));
      PUT(HDRP(NEXT_BLKP(ptr)), PACK(0, 1));
      return ptr;
    }
    else if (!GET_ALLOC(HDRP(next)) && old_size + next_size >= asize) { /* 与下一个块合并 */
      freelist_remove(next);

      PUT(HDRP(ptr), PACK(old_size + next_size, 1));
      PUT(FTRP(ptr), PACK(old_size + next_size, 1));
      return ptr;
    }
    else if (!GET_ALLOC(HDRP(prev)) && old_size + prev_size >= asize) { /* 与前一个块合并 */
      freelist_remove(prev);

      PUT(HDRP(prev), PACK(old_size + prev_size, 1));
      PUT(FTRP(prev), PACK(old_size + prev_size, 1));
      memmove(prev, ptr, old_size - WSIZE);
      return prev;
    }
    else if (!GET_ALLOC(HDRP(prev)) && !GET_ALLOC(HDRP(next)) && sum_size >= asize) { /* 与前后两个块一同合并 */
      freelist_remove(prev);
```

```
      freelist_remove(next);

      PUT(HDRP(prev), PACK(sum_size, 1));
      PUT(FTRP(prev), PACK(sum_size, 1));
      memmove(prev, ptr, old_size - WSIZE);
      return prev;
    }
    /* 合并块的大小不足以使用 */
    newptr = mm_malloc(size);
    memmove(newptr, ptr, old_size - WSIZE);
    mm_free(ptr);
    return newptr;
  }
}

```

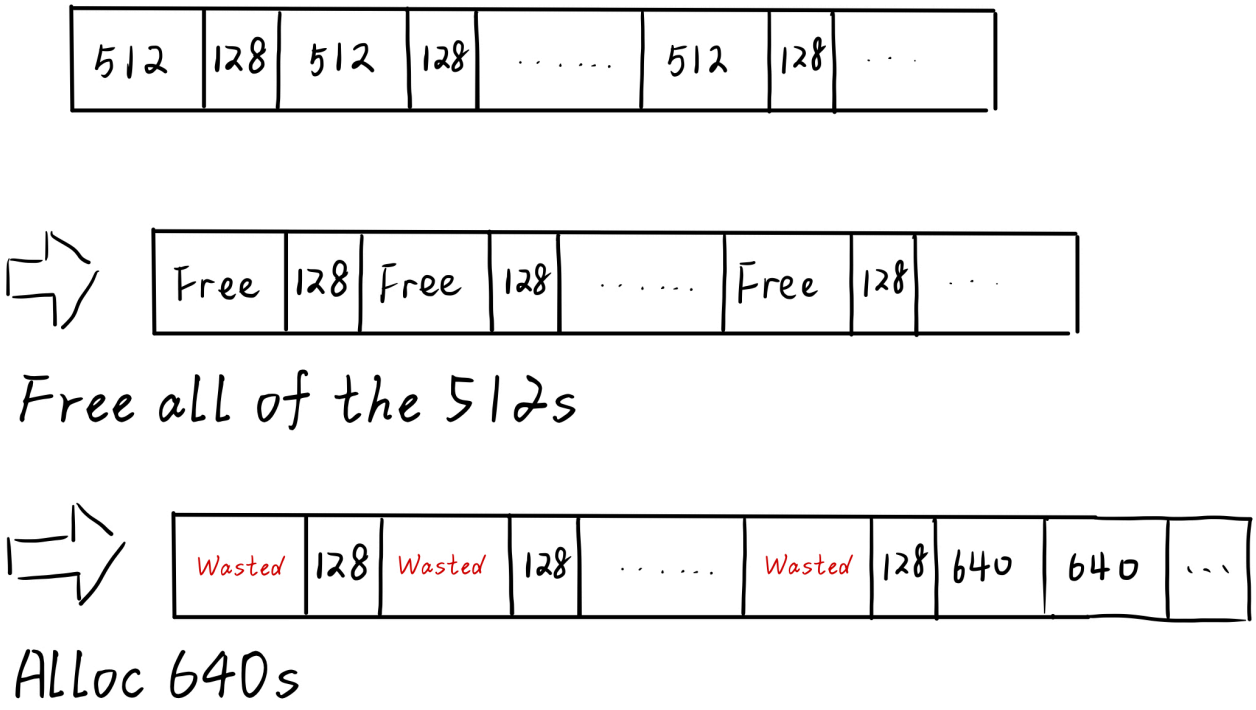
更改等价类的划分/CHUNKSIZE

最终程序将大小划分成了13个等价类，与上面所述的略有不同。划分得过于细，或过于宽都将导致效率倒退。最终的划分结果是多次调试后决定。

CHUNKSIZE的大小也是如此，太大可能会导致空间浪费，而太小则会导致效率低下。

针对数据的改进：binary.rep

在多次测试中，binary.rep的空间利用率一直非常很低。观察数据，发现该测例是如下图运作的：

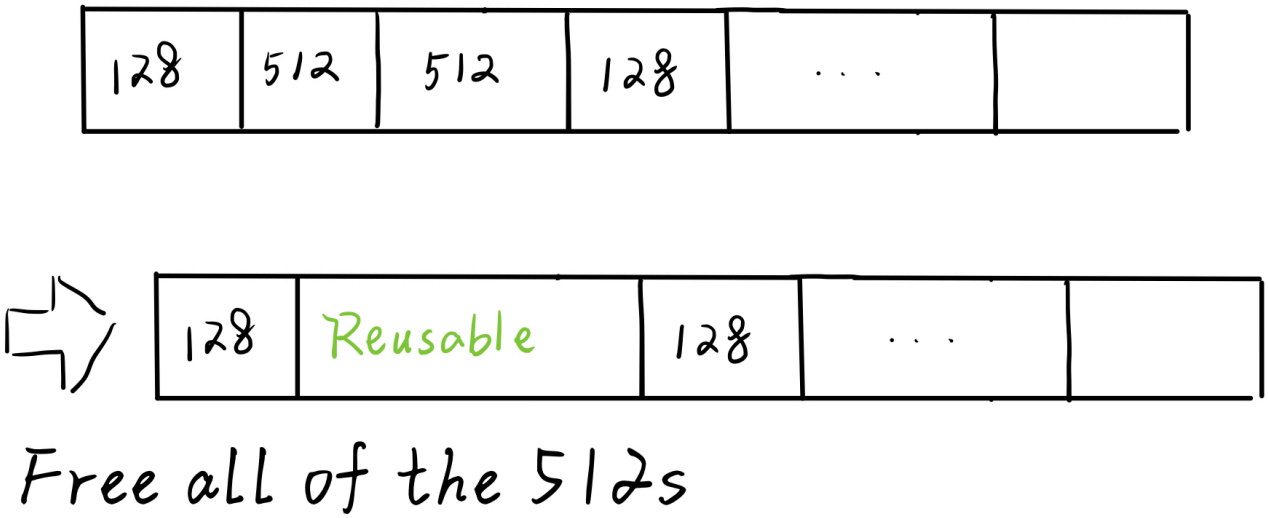


第7頁，共7頁

即：外部碎片过于繁多，且他们相互之间不能合并，最终导致了巨大的内存浪费。

针对这种测例，采用的方法是：在分割块时取前块和后块交替的进行。为此，程序维护了一个全局变量`_rand`，用于记录当前应当把分割出来的块放在前面还是放在后面。

改进过后的示意图：



不过，这仅仅只能做到“优化”而不是彻底解决外部碎片带来的问题。这种情况下，分配器仍然浪费了$1024-640=384$字节的空间。相比于前，只是少浪费了一些空间而已。

实验结果

实验环境

本地的测试环境为mac OS，芯片为M2芯片。

在服务器上的测试即为课程所提供的服务器。

测试结果

- 本地

```
Results for mm malloc:
trace              name      valid  util      ops       secs    Kops
 1      amptjp-bal.rep        yes   99%     5694  0.000229   24886
 2        cccp-bal.rep        yes   99%     5848  0.000181   32327
 3     cp-decl-bal.rep        yes   99%     6648  0.000185   35935
 4        expr-bal.rep        yes   99%     5380  0.000110   48820
 5 coalescing-bal.rep         yes   98%    14400  0.000138  104423
 6      random-bal.rep        yes   96%     4800  0.000396   12137
 7     random2-bal.rep        yes   96%     4800  0.000352   13621
 8      binary-bal.rep        yes   83%    12000  0.000141   84866
 9     binary2-bal.rep        yes   76%    24000  0.000534   44978
10     realloc-bal.rep        yes  100%    14401  0.000110  131396
11    realloc2-bal.rep        yes   84%    14401  0.000122  117751
Total                              93%   112372  0.002498   44992

Score = (56 (util) + 40 (thru)) * 11/11 (testcase) = 96/100
wangjuanli@wangjuanlideMacBook-Air malloclab-handout %
```

- 服务器：

```
trace              name      valid  util      ops       secs    Kops
 1      amptjp-bal.rep        yes   99%     5694  0.000290   19607
 2        cccp-bal.rep        yes   99%     5848  0.000294   19878
 3     cp-decl-bal.rep        yes   99%     6648  0.000348   19109
 4        expr-bal.rep        yes   99%     5380  0.000257   20918
 5 coalescing-bal.rep         yes   98%    14400  0.000263   54732
 6      random-bal.rep        yes   96%     4800  0.001258    3816
 7     random2-bal.rep        yes   96%     4800  0.001162    4132
 8      binary-bal.rep        yes   83%    12000  0.000562   21349
 9     binary2-bal.rep        yes   76%    24000  0.002539    9454
10     realloc-bal.rep        yes  100%    14401  0.000317   45429
11    realloc2-bal.rep        yes   84%    14401  0.000398   36229
Total                              93%   112372  0.007687   14618

Score = (56 (util) + 40 (thru)) * 11/11 (testcase) = 96/100
u2021012957@hp:~/test/malloclab-handout$
```

二者仅在吞吐率上存在差异，差距大约为3倍左右。

实验总结

- 事实上，虽然最终的实现是分离适配 + best fit，但基本的框架仍是基于CSAPP提供的隐式空闲链表实现方式，改动的地方实际只占少数。这是因为CSAPP上提供的合并(coalesce)与放置(place)函数对于不同的方式足够的适用，且接下来的两种方法也都需要频繁地调用这两个函数。故总体而言，**从隐式空闲链表至最终版本花费的精力不如从零至理解隐式空闲链表**。若要完整地体验malloc lab，从CSAPP书上给出的样例实现入手是再好不过的。
- 除此以外，在这次实验中，我对动态内存分配有了新的理解，特别是外部碎片的部分。在解决binary.rep的问题时，我曾上网搜索解决相关问题的方法得到了一些启发：

    - 一方面，像这次实验中组织数据结构的方式必然带来外部碎片的影响。**这是由数据结构决定的**：我们不能移动一个已分配块的物理位置，否则，程序端所有指向该位置的指针（该地址的引用）都需要改变。这是不切实际的。
    - 另一方面，**分页**可以作为解决外部碎片过多的途径。正如binary.rep体现的规律一样，当外部碎片过多，且大小不足以分配给新的请求时，便会造成空间的浪费。而分页管理可以使得内存被分为一个一个相互独立的页，每个页面中的外部碎片大小都得到控制，总计下来，进程的外部碎片数量和大小便得到了控制。这种处理方式实际上在课堂上也有所体现，并且是值得我学习的。
- 最后，malloc lab带给我的感觉与设计哈希函数及冲突解决策略非常相像。分配一个新的内存块即对应于哈希映射，而找寻/释放内存块的过程与冲突解决的过程也有异曲同工之妙。不同的是，malloc lab中样例的苛刻程度可能更加的高。若没有与周围同学一齐交流，有很多优化的方式我未必能够想到。

交流记录

在与戴炜楠2021012958的交流过程中，他启发了我如何设计内存块的数据结构，以及进一步优化`mm_realloc`的方法（如果块位于堆顶）。

CMU中的Malloc lab似乎是多人合作完成。如果同学之间能有更多的优化技术交流，实验的效果可能能够更好。